

Understanding the I²C Protocol From First Principles

SDA — SCL

How chips communicate using only two wires

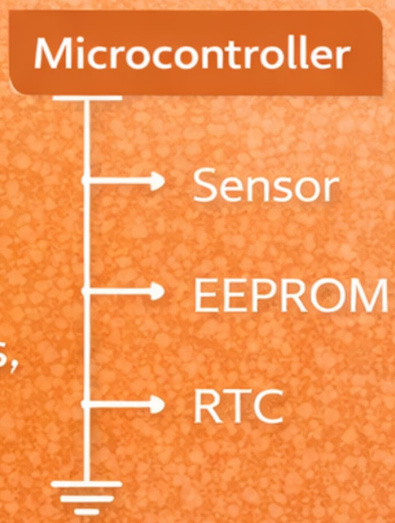
Pushpa Nagashree | VLSI Learner

Why do we need a protocol like I²C?

Communication between chips usually requires separate wires for each device.

Example:

- Microcontroller ⟶ Sensor
- Microcontroller ⟶ EEPROM
- Microcontroller ⟶ RTC



Each connection needɔ its own wires,

- Wiring becomes complex
- More pins are required on the chip
- PCB design becomes harder
- Cost increases

Is there a simpler way for multiple devices to communicate?

Pushpa Nagashree | VLSI Learner

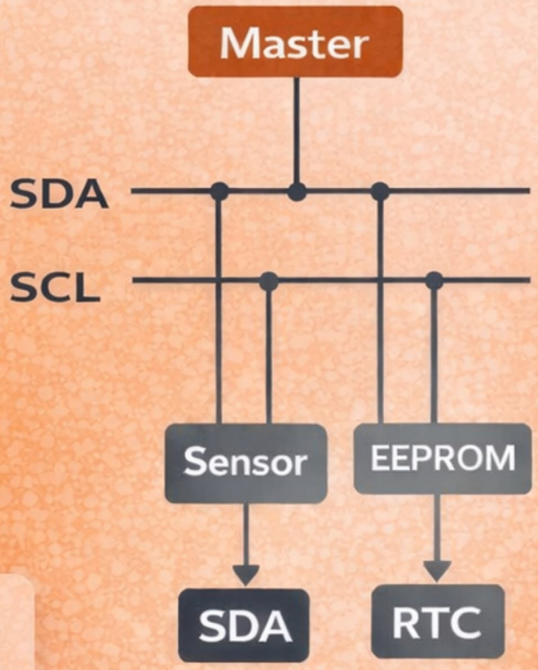
The I²C Bus: A Simpler Solution

Instead of using separate wires for every device,

I²C allows multiple devices to share the same bus.

Only **two** wires are needed:

- **SDA** – Serial Data Line
- **SCL** – Serial Clock Line



All devices connect to the same two lines.

But if all devices **share the same wires**, how do they **avoid** signal conflicts?

Pushpa Nagashree | VLSI Learner

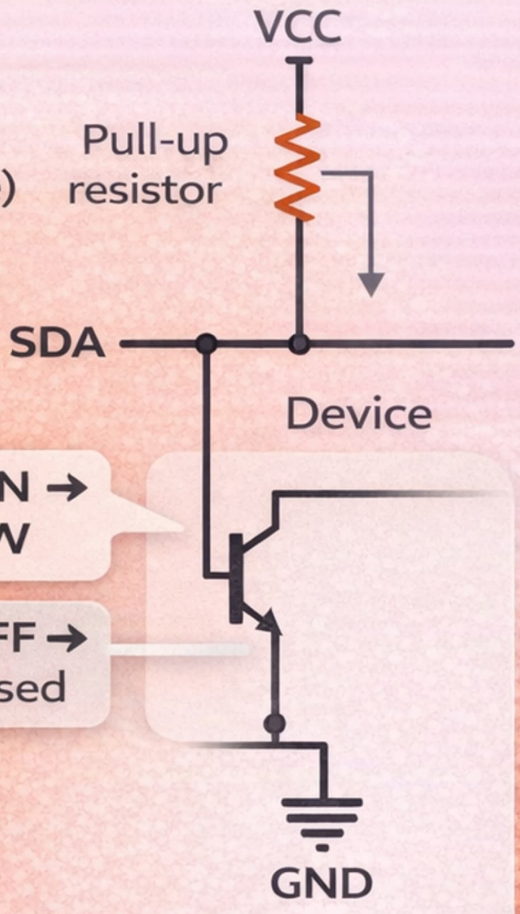
Open-Drain: The Secret Behind I²C

In an I²C bus, devices do not drive HIGH.

Each device can only:

- **Pull the line LOW**
- **Release the line (High-Impedance)**

Pull-up resistors bring the line HIGH when no device is pulling it LOW.



LOW dominates the bus.

Pushpa Nagashree | VLSI Learner

How communication begins and ends

START condition

- SDA: HIGH → **LOW** while SCL is HIGH



STOP condition

- SDA: **LOW** → **HIGH** while SCL is HIGH



Important rule:

SDA changes only when SCL is LOW
(except START and STOP)

Pushpa Nagashree | VLSI Learner

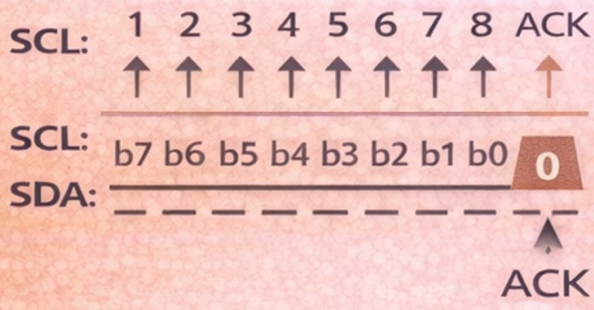
How Devices Confirm Reception

Each byte in I²C uses **9 clock pulses.**

- Clock 1–8 → Data bits
- Clock 9 → **Acknowledgement**

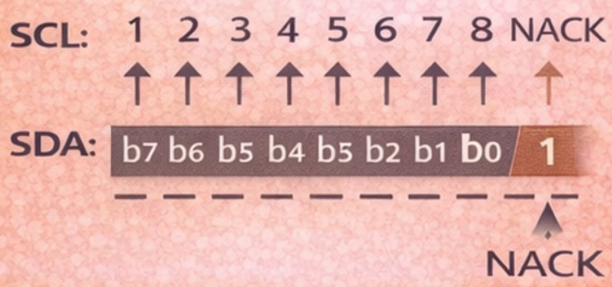
ACK (Acknowledgment)

- Receiver pulls SDA **LOW** during the 9th clock pulse.
- **LOW** → Data received successfully



NACK (Not Acknowledged)

- Receiver leaves SDA HIGH (no acknowledgement)



Important rule:

Transmitter releases SDA ▸ Receiver controls the ACK bit

Pushpa Nagashree | VLSI Learner

How the Master Selects a Device

All devices share the same SDA and SCL lines.

To communicate with a specific device, the master sends an address.

Only the device with the matching address responds.

7-Bit Addressing
(Most Common)

All devices share the same SDA and SCL lines.

To communicate with a specific device, the master sends an address.

A6	A5	A4	A3	A2	A1	R/W

- 7 bits → Slave address
- 1 bit → Read / Write

- R/W = 0 → Write
- R/W = 1 → Read

10-Bit Addressing
(Extended Mode)

10-bit addressing allows more devices to share the I²C bus,

First byte:

1	1	1	1	0	A9	A8	R/W

Second byte:

A7	A6	A5	A4	A3	A2	A1	A0

First byte → 11110 + upper address bits

Second byte → remaining address bits

START

↓

Address + R/W

↓

ACK

Important note:

7-bit addressing → most commonly used

10-bit addressing → used when more devices share the bus

Pushpa Nagashree | VLSI Learner

Complete i²C Communication

WRITE TRANSACTION

START
↓
Slave Address + W
↓
ACK
↓
Data Byte
↓
ACK
↓
Data Byte
↓
ACK
↓
STOP

Master writes data to the slave.
Slave acknowledges each byte.

READ TRANSACTION

START
↓
Slave Address + R
↓
ACK
↓
Slave sends Data Byte
↓
Master ACK
↓
Slave sends Data Byte
↓
Master NACK
↓
STOP

During read operation,
the slave transmits data
and the master sends ACK/NACK.

Every byte transfer: 8 data bits + 1 ACK bit

Write → Master sends data
Read → Slave sends data

Pushpa Nagashree | VLSI Learner